

1 . To Create Process Escape Sequences, Handle Escaped Spaces, Escape Special Symbols, Preserve Characters, Validate Parser Output, Test Complex Inputs.

Aim : To create and test process escape sequences in Ubuntu, handle escaped spaces, escape special symbols, preserve characters, validate parser output, and test complex inputs.

Program :

```
GNU nano 7.2
#include <stdio.h>

int main()
{
    printf("Process Escape Sequence Demonstration\n\n");

    printf("1. Normal text:\n");
    printf("Hello World\n\n");

    printf("2. Escaped space:\n");
    printf("Hello\\ World\n\n");

    printf("3. Escaped dollar symbol:\n");
    printf("\\$HOME\n\n");

    printf("4. Escaped star symbol:\n");
    printf("\\*\n\n");

    printf("5. Escaped question mark:\n");
    printf("\\?\n\n");

    printf("6. Escaped semicolon:\n");
    printf("Hello\\;World\n\n");

    printf("7. Escaped pipe symbol:\n");
    printf("Hello\\|World\n\n");

    printf("8. Escaped ampersand:\n");
    printf("Hello\\&World\n\n");

    printf("9. Complex input:\n");
    printf("Hello\\ World\\ \\$USER\\ \\*\\ \\?\\ \\&\\ \\;\\ \\|\\n\n");

    printf("10. Quoted text:\n");
    printf("'Hello $USER * ? & ; |'\n");

    return 0;
}
```

```
GNU nano 7.2
#!/bin/bash

echo "==== PROCESS ESCAPE SEQUENCE TEST ====="

echo "Test 1: Escaped Space"
echo Hello\ World

echo "Test 2: Escaped Dollar"
echo \$HOME

echo "Test 3: Escaped Star"
echo \*

echo "Test 4: Escaped Question Mark"
echo \?

echo "Test 5: Escaped Semicolon"
echo Hello\;World

echo "Test 6: Escaped Pipe"
echo Hello\|World

echo "Test 7: Escaped Ampersand"
echo Hello\&World

echo "Test 8: Single Quotes"
echo '$HOME * ? & ; |'

echo "Test 9: Double Quotes"
echo "$HOME"

echo "Test 10: Complex Input"
echo Hello\ World\ \$USER\ *\ ?\ &\ ;\ \|

echo "==== TEST COMPLETED ====="
```

Output :

```
navaneeth@NAVANEETH: ~/we X + v
navaneeth@NAVANEETH:~$ mkdir week6
navaneeth@NAVANEETH:~$ cd week6
navaneeth@NAVANEETH:~/week6$ pwd
/home/navaneeth/week6
navaneeth@NAVANEETH:~/week6$ nano escape.c
navaneeth@NAVANEETH:~/week6$ gcc escape.c -o escape
navaneeth@NAVANEETH:~/week6$ ls
escape  escape.c
navaneeth@NAVANEETH:~/week6$ ./escape
Process Escape Sequence Demonstration

1. Normal text:
Hello World

2. Escaped space:
Hello\ World

3. Escaped dollar symbol:
\${HOME}

4. Escaped star symbol:
\*

5. Escaped question mark:
\?ëïo\\World

6. Escaped semicolon:
Hello\;World

7. Escaped pipe symbol:
Hello\\|World

8. Escaped ampersand:
Hello\&World

9. Complex input:
Hello World \\$USER\ \*\ \?\ \&\ \;\ \

10. Quoted text:
'Hello $USER * ? & ; |'
navaneeth@NAVANEETH:~/week6$
```

```
navaneeth@NAVANEETH: ~/we -X
'Hello $USER * ? & ; |'
navaneeth@NAVANEETH:~/week6$ echo Hello\ World
Hello World
navaneeth@NAVANEETH:~/week6$ echo \$HOME
$HOME
navaneeth@NAVANEETH:~/week6$ echo $HOME
/home/navaneeth
navaneeth@NAVANEETH:~/week6$ echo \*
*
navaneeth@NAVANEETH:~/week6$ echo \?
?
navaneeth@NAVANEETH:~/week6$ echo Hello\;World
Hello;World
navaneeth@NAVANEETH:~/week6$ echo Hello\\World
Hello\World
navaneeth@NAVANEETH:~/week6$ echo Hello&World
Hello&World
navaneeth@NAVANEETH:~/week6$ echo '$HOME'
$HOME
navaneeth@NAVANEETH:~/week6$ echo 'Hello $USER * ? & ; |'
Hello $USER * ? & ; |
navaneeth@NAVANEETH:~/week6$ echo "Hello World"
Hello World
navaneeth@NAVANEETH:~/week6$ echo "Hello $USER"
Hello navaneeth
navaneeth@NAVANEETH:~/week6$ echo "Hello $USER"
Hello navaneeth
navaneeth@NAVANEETH:~/week6$ echo '$HOME'
$HOME
navaneeth@NAVANEETH:~/week6$ echo Hello\ World\ \$USER\ \*\ \?\ \&\ \;\ \|
Hello World $USER * ? & ; |
navaneeth@NAVANEETH:~/week6$ nano parser_test.sh
navaneeth@NAVANEETH:~/week6$ chmod +x parser_test.sh
navaneeth@NAVANEETH:~/week6$ ls -l parser_test.sh
-rwxr-xr-x 1 navaneeth navaneeth 594 Sep 18 13:33 parser_test.sh
navaneeth@NAVANEETH:~/week6$ ./parser_test.sh
===== PROCESS ESCAPE SEQUENCE TEST =====
Test 1: Escaped Space
Hello World
Test 2: Escaped Dollar
$HOME
Test 3: Escaped Star
```

```

navaneeth@NAVANEETH: ~/week6
*
Test 4: Escaped Question Mark
?
Test 5: Escaped Semicolon
Hello;World
Test 6: Escaped Pipe
Hello|World
Test 7: Escaped Ampersand
Hello&World
Test 8: Single Quotes
$HOME * ? & ; |
Test 9: Double Quotes
/home/navaneeth
Test 10: Complex Input
Hello World $USER * ? & ; |
===== TEST COMPLETED =====
navaneeth@NAVANEETH:~/week6$ ./parser_test.sh | grep "Hello World"
Hello World
Hello World $USER * ? & ; |
navaneeth@NAVANEETH:~/week6$ ./parser_test.sh | grep '\$HOME'
$HOME
$HOME * ? & ; |
navaneeth@NAVANEETH:~/week6$ ./parser_test.sh | grep '\*'
*
$HOME * ? & ; |
navaneeth@NAVANEETH:~/week6$ ./parser_test.sh | grep '\?'
?
$HOME * ? & ; |
Hello World $USER * ? & ; |
navaneeth@NAVANEETH:~/week6$ touch my\ file.txt
navaneeth@NAVANEETH:~/week6$ ls
escape  escape.c  "my file.txt"  parser_test.sh
navaneeth@NAVANEETH:~/week6$ ls my\ file.txt
my file.txt
navaneeth@NAVANEETH:~/week6$ ls "my file.txt"
my file.txt
navaneeth@NAVANEETH:~/week6$ touch 'student file.txt'
navaneeth@NAVANEETH:~/week6$ ls
escape  escape.c  "my file.txt"  parser_test.sh  'student file.txt'
navaneeth@NAVANEETH:~/week6$ ls 'student file.txt'
student file.txt
navaneeth@NAVANEETH:~/week6$ █

```

2 . To Create Child Processes, Execute Programs, Handle Execution Errors, Pass Arguments, Manage Parent Process, Test Command Launching.

Aim : To create a child process using `fork()` , execute another program using `exec()` , handle execution errors, pass arguments to the executed program, manage the parent process, and test command launching in Ubuntu.

Program :

```
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main()
{
    pid_t pid;

    printf("Parent process started.\n");
    printf("Parent PID: %d\n", getpid());

    pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }

    if (pid == 0)
    {
        // Child process
        printf("\nChild process created successfully.\n");
        printf("Child PID: %d\n", getpid());
        printf("Parent PID: %d\n", getppid());

        printf("\nExecuting ls command using execlp()...\n");
        execlp("ls", "ls", "-l", NULL);

        // This executes only if execlp() fails
        perror("execlp failed");
        exit(1);
    }
    else
    {
        // Parent process
        printf("\nParent is waiting for child process...\n");
        wait(NULL);

        printf("\nChild process completed.\n");
        printf("Parent process completed.\n");
    }

    return 0;
}
```

```
navaneeth@NAVANEETH: ~/we × + ∨  
navaneeth@NAVANEETH:~$ mkdir week6_process  
navaneeth@NAVANEETH:~$ cd week6_process  
navaneeth@NAVANEETH:~/week6_process$ pwd  
/home/navaneeth/week6_process  
navaneeth@NAVANEETH:~/week6_process$ nano process.c  
navaneeth@NAVANEETH:~/week6_process$ gcc process.c -o process  
navaneeth@NAVANEETH:~/week6_process$ ls  
process  process.c  
navaneeth@NAVANEETH:~/week6_process$ ./process  
Parent process started.  
Parent PID: 535  
  
Parent is waiting for child process...  
  
Child process created successfully.  
Child PID: 536  
Parent PID: 535  
  
Executing ls command using execlp()...  
total 20  
-rwxr-xr-x 1 navaneeth navaneeth 16304 Sep 18 13:46 process  
-rw-r--r-- 1 navaneeth navaneeth 968 Sep 18 13:45 process.c  
  
Child process completed.  
Parent process completed.
```

Output :